SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE CODE: CSA04
COURSE NAME: Operating Systems

COURSE OUTCOME COVERED

CO1: *Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.*

QUESTIONS: 5 **Total Marks:100**

Annexure A

SHORT ANSWER TEST

Code of Conduct:

I VISHAL BALAJI.J (192525014 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

90/100

ASSESSMENT TOOL 1 – SHORT ANSWER TEST

COURSE NAME: OPERATING SYSTEMS

COURSE CODE: CSA04 | COURSE OUTCOME: CO1

Submitted by: Vishal Balaji J | Register Number: 192525014

CO1: Apply the concepts of Operating System types, structures, and system calls to demonstrate their functions in various computing environments.

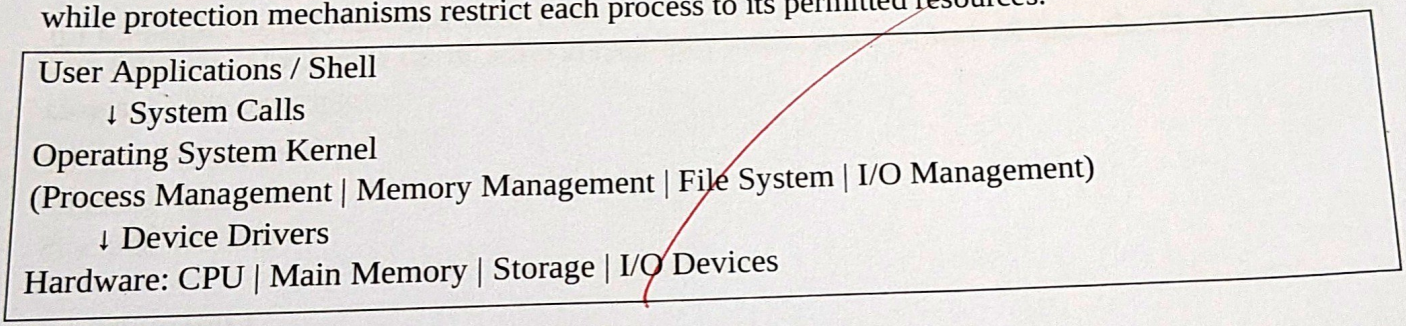
Question 1 – Operating System Types, Structure and Process Creation

(a) Types of Operating Systems and justification

A modern computer system that supports several users and processes needs an operating system capable of sharing resources fairly and securely. A multi-user operating system allows more than one user to access the computer's services, while a multitasking operating system switches the CPU among multiple processes so that several applications appear to run at the same time. A time-sharing system gives each active process a small CPU time slice, improving responsiveness. For a server environment, a multiprocessor operating system can also use more than one CPU core to execute work in parallel. Therefore, a multi-user, multitasking, time-sharing OS is a suitable choice for the stated scenario.

(b) Operating System structure and resource management

The operating system acts as an intermediary between user applications and the hardware. Applications request services through system calls. The kernel manages process scheduling, memory allocation, file access, and input/output devices. Device drivers communicate with the hardware, while protection mechanisms restrict each process to its permitted resources.



(c) System calls to create and execute a process

On a Unix-like system, a common sequence is fork(), exec(), and wait(). The fork() system call creates a child process. The child then calls an exec-family function to replace its process image with the requested program. The parent may call wait() or waitpid() to wait for the child to finish.

Example sequence:

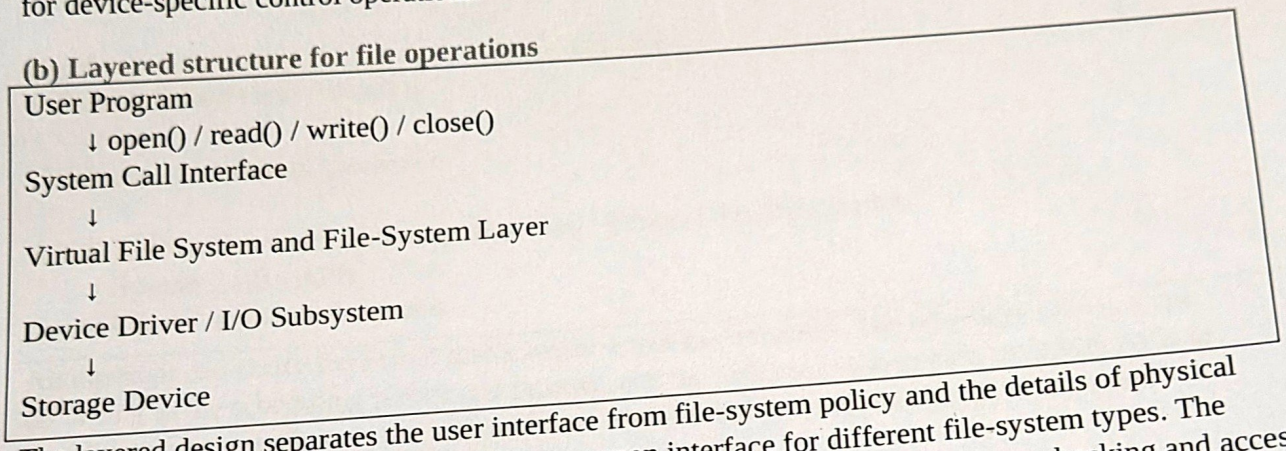
```
pid_t pid = fork();
if (pid == 0) {
   execl("/bin/ls", "ls", "-l", (char *)NULL);
   _exit(1);  // reached only if exec fails
} else if (pid > 0) {
   waitpid(pid, NULL, 0);
}
```

Question 2 – File and Device Management System Calls

(a) Categories of system calls

System calls provide a controlled interface through which a program requests operating-system services. File-management calls create, open, read, write, reposition and close files. Examples include open(), read(), write(), lseek(), close(), unlink() and stat(). Device-management calls request access to devices, perform input/output operations, obtain device information and release resources. On Unix-like systems, read() and write() can operate on device file descriptors, while ioctl() is commonly used for device-specific control operations.

(b) Layered structure for file operations



The layered design separates the user interface from file-system policy and the details of physical storage. The virtual file system provides a common interface for different file-system types. The driver and I/O subsystem translate requests into device-specific operations. Error checking and access permissions are applied along the path.

(c) Example: open, read and close a file

The following C example opens a file in read-only mode, reads a block of data, checks the result, and closes the file descriptor.

```
#include <fcntl.h>
#include <unistd.h>
#include <stdio.h>

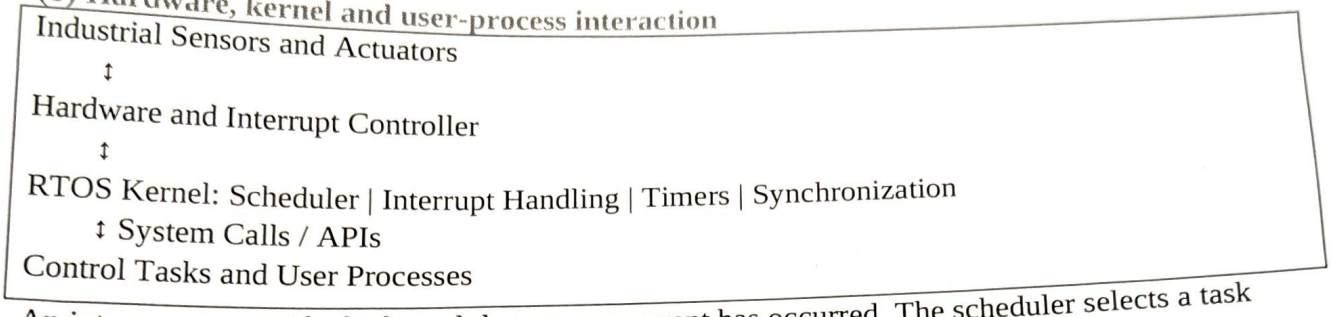
int main(void) {
  char buffer[128];
  int fd = open("input.txt", O_RDONLY);
  if (fd == -1) { perror("open"); return 1; }
  ssize_t n = read(fd, buffer, sizeof(buffer));
  if (n == -1) { perror("read"); close(fd); return 1; }
  if (n > 0) write(STDOUT_FILENO, buffer, (size_t)n);
  if (close(fd) == -1) { perror("close"); return 1; }
  return 0;
}
```

Question 3 – Real-Time Operating System

(a) OS type and suitability

A real-time operating system (RTOS) is designed to respond to events within predictable time limits. In industrial machinery, a delayed response may affect safety or product quality. A hard real-time system treats missing a critical deadline as unacceptable, whereas a soft real-time system can tolerate occasional delays with reduced quality of service. A hard or safety-oriented RTOS is suitable when control actions, such as stopping a machine or responding to a sensor threshold, must occur within a specified deadline.

(b) Hardware, kernel and user-process interaction



An interrupt can notify the kernel that a sensor event has occurred. The scheduler selects a task according to the scheduling policy and priority, and the task uses system calls or RTOS APIs to coordinate access to shared data and hardware. Predictable interrupt handling and bounded synchronization delays are important for meeting deadlines.

(c) System calls for process synchronization

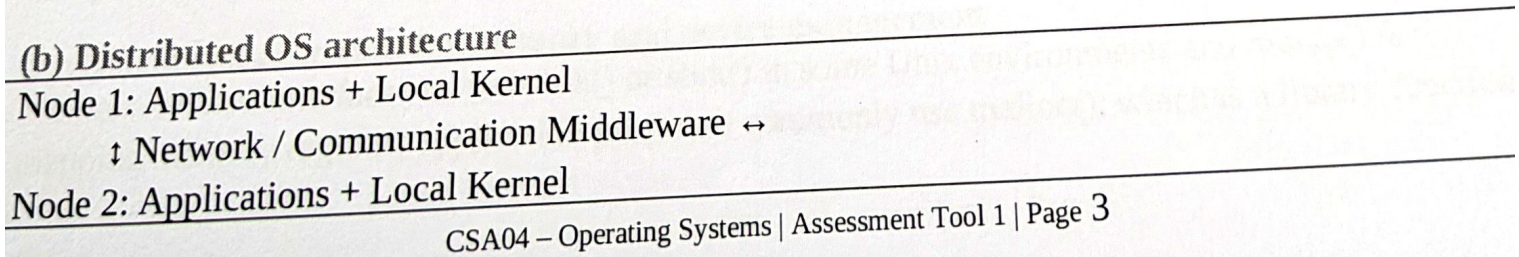
Synchronization prevents concurrent tasks from corrupting shared data or accessing a resource at the same time. The exact names vary by RTOS, but common operations include semaphore take/give, mutex lock/unlock, condition-variable wait/signal, event wait/set, and message-queue send/receive. A task may block on a semaphore until an interrupt-handling task signals that data is ready. A mutex can protect a shared data structure, while a message queue can pass sensor readings safely between tasks. Priority inheritance may reduce priority inversion when a lower-priority task holds a lock needed by a higher-priority task.

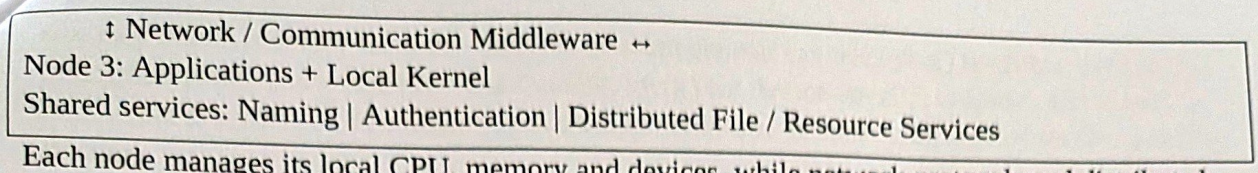
Question 4 – Distributed Operating Systems and IPC

(a) Distributed OS and comparison with centralized OS

A distributed operating system coordinates multiple networked computers and aims to make their resources appear integrated to users or applications. It can support resource sharing, workload distribution and fault tolerance, although the degree of transparency varies by system. In a centralized operating system, processing and resource management are mainly controlled by one computer. A distributed environment can provide scalability and continue some services when a node fails, but it also introduces network delays, partial failures, synchronization challenges and security concerns.

(b) Distributed OS architecture





Each node manages its local CPU, memory and devices, while network protocols and distributed services enable communication and sharing. A distributed system must account for messages that arrive late or fail, nodes that become unreachable, and consistency when multiple users access shared data.

(c) System calls used in inter-process communication

For communication between processes on the same machine, common Unix mechanisms include pipe() for a byte stream, socketpair() for connected local sockets, and shared memory APIs for exchanging data through a common memory region. For network communication, socket(), bind(), listen(), accept(), connect(), send() and recv() are typical calls. Synchronization may use semaphores or mutexes. The required sequence depends on the IPC mechanism. For a client-server socket, the server commonly creates a socket, binds and listens, then accepts a connection, while the client creates a socket and connects.

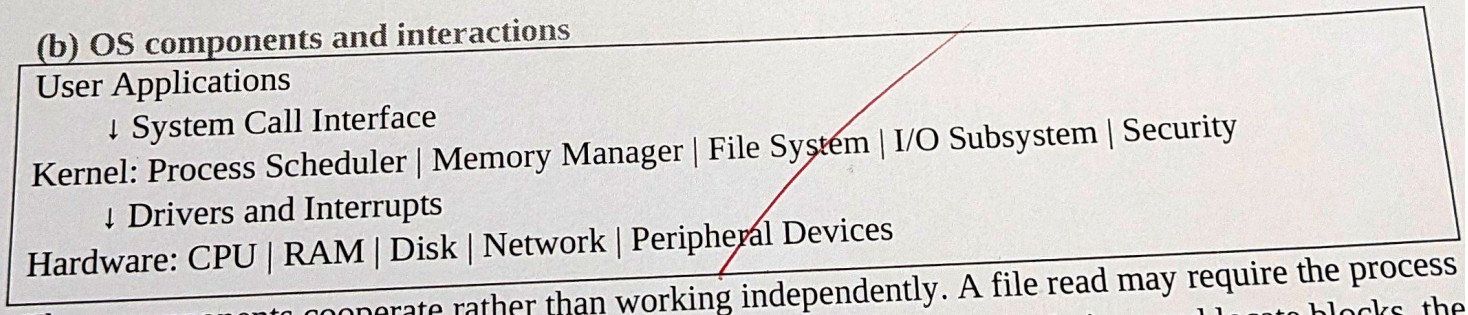
Example socket sequence: Server: socket() → bind() → listen() → accept() → recv()/send() → close(). Client: socket() → connect() → send()/recv() → close().

Question 5 – Core OS Functions and Resource Management

(a) Major functions

The operating system coordinates the resources needed to run programs. Process management creates and terminates processes, schedules CPU time, supports synchronization and handles communication. Memory management allocates and releases memory, protects address spaces and may use virtual memory to extend the usable address space. Device management coordinates input/output requests through drivers, buffering, interrupts and device queues. The file system organizes persistent data and controls permissions. Security, error handling, accounting and user interfaces support reliable operation across the system.

(b) OS components and interactions



These components cooperate rather than working independently. A file read may require the process scheduler to run the requesting process, the file system to validate permissions and locate blocks, the memory manager to provide a buffer, and the I/O subsystem to ask the storage driver to retrieve data. The process may wait until the operation completes and then resume.

(c) System call examples for memory and device management

Memory-related interfaces include brk() or sbrk() in some Unix environments and mmap() for mapping memory regions; application programs commonly use malloc(), which is a library functi

that may obtain memory from the OS. Device management can use open() to obtain a device file descriptor, read() and write() for data transfer, ioctl() for device-specific controls, and close() to release the descriptor. The availability and exact behaviour of these calls depend on the operating system.

Example sequence: fd = open("/dev/example", O_RDWR); → ioctl(fd, REQUEST, &value); → read(fd, buffer, size) or write(fd, buffer, size); → close(fd). The device name and ioctl request must be valid for the target system.

Conclusion

Operating systems provide the structures and system-call interfaces required to manage processes, memory, files, devices and communication. Multi-user and time-sharing systems support concurrent work, an RTOS supports predictable responses, and distributed systems coordinate resources across connected nodes. Understanding the relationship between applications, system calls, kernel components and hardware helps explain how an OS provides efficient and controlled access to shared resources.